

Nova, fully by voice: native computer use with Jev

Research · 28 September 2026 · no code yet

Update, later on 28 September. After this was written, Reflex caught up with Jev on understanding commands:

- 84% right on the hardest phrasings, against Jev's 86%;
- about 2.7 ms a decision, against Jev's 470 ms;
- on the Mac, and free.

So the plan (*Nova_Computer_Use_Plan.pdf*) builds computer use Reflex-first:

- Direct commands, app adapters, and on-screen steps named in your words need no Jev.
- Jev, or a local model, comes in only for the hard middle: unfamiliar pages and unexpected screens. Even that happens only if measurement shows it's needed.

Read the Jev-driven loop in section 5 as an option for that middle, not the default. The rest is unchanged.

The short answer: yes, most of it is possible, and it can be fast. The way to get there is not a faster model looking at screenshots. It is doing far less thinking per command:

- **Most commands never need a loop.** "Open Chrome and go to github.com" is two native actions. Reflex decides them in about 2 ms, and Nova can carry them out in well under a second, plus the page's own loading time. Today the request goes to Claude, which takes a screenshot and thinks at every step.
- **Everything else runs in Nova's own loop.**
 - Nova Eyes reads the screen through Accessibility: the list of buttons, fields and links macOS already keeps for every app.
 - Jev picks the next step from a typed menu built from that list.
 - Nova's own hands carry it out.

That's roughly **0.6–1 s a step**, against the 2–5+ s a step Claude takes.

- **What Nova has done once, it can replay.** A learned routine runs natively and asks Jev only where the screen looks different from last time.
- **Claude can leave computer use.** What's lost is open-ended reasoning over unfamiliar pages, and writing long text. A small local model can do the writing when you ask, and Nova can ask you when it's unsure.

Piece of the path	Today	Target
Deciding what a command means	Reflex ≈ 2 ms · Jev ≈ 470 ms (p50), 520 ms (p90)	Same: Reflex first, Jev when it's needed
One step on the screen	Claude: a screenshot and a think, 2–5+ s	Jev loop: ≈ 0.6 –1.0 s
"Open Chrome and go to github.com"	Several Claude steps: seconds to tens of seconds	Under 1 s, plus the page's load time

Measured on this Mac: Reflex and Jev timings from the Reflex evaluation on 27 September (about 2000 Jev calls). Everything else on this page is an estimate to be measured before building. See section 12.

The main risks:

- Apps that don't describe themselves to Accessibility: games, canvas apps such as Figma's canvas, some Electron apps.
- Jev's limits:
 - It can't write text.
 - At most 255 options per choice.
 - At most 32k of state.
 - About 0.5 s per call over the network.
 - Screen text leaves the Mac on every call.
- Long tasks that need judgement.
- Wrong actions from misheard words.

1. What "full control by voice" covers

Area	You say	How Nova would do it natively	Today
Apps and windows	open, quit, switch to, hide, put side by side, full screen	Nova Eyes and hands; Spaces through shortcuts	Mostly there
Browser: getting around	"go to github", "search for ...", back, reload, new tab, close tab, next tab	A browser adapter: addresses and tabs through Apple Events, plus keyboard shortcuts. Site names become addresses in code.	Missing: the biggest speed win
Browser: inside pages	"click Pull requests", "scroll to the comments", "put my email in the email field"	The page's Accessibility list, or a browser extension later	Claude loop
Menus and dialogs	"File, Export as PDF", "press Save", "cancel"	Menu items by name through Accessibility; a dialog's default and cancel buttons	Partly (ui_control)
Typing and editing	"type ...", "new line", "delete that word", "undo"	A dictation grammar in code, plus keys	Partly
Files	open, move, rename, put in the Trash	Hands	There
System and media	volume, brightness, Bluetooth, lock, a Settings pane, play, next	Hands; Settings panes by link	Mostly there
Several steps at once	"open Chrome, go to GitHub and open my Agentic_OS pull requests"	Split into steps; each goes to an adapter or the loop	Claude loop

2. Why it's slow today

The path of "open Chrome and go to a page":

1. **You stop speaking.** Nova waits for the end of the turn: about 0.24 s for the speech detector, then 0.4 s of patience after a finished sentence (longer when you trail off). Parakeet then transcribes, in a fraction of a second.
2. **Reflex or Jev decides what it means.** There's no command for opening a web address, so the request becomes `computer_task`.
3. **`computer_task` hands the whole job to a brain, Claude.** Claude drives the screen through Nova Eyes one tool at a time: look, click the address bar, type, press Return, look again. It thinks before every step, and each round trip takes seconds.

So the time goes to two things: **a missing direct command**, and **a thinking model in the loop for mechanical steps**. Both can go.

What Nova already has to build on

- **Nova Eyes:**
 - the Accessibility list, with an id per element;
- **Jev:** typed choices, well calibrated, about 0.5 s per call, about \$0.0002 per call.

- screenshots, and text read from them on the Mac;
- click, type, keys, scroll and drag;
- a highlight frame, and knowing whether you're typing.
- **Hands:** system, media, windows, files, clipboard, Shortcuts, and "click send" by name.
- **Reflex:** about 2 ms, on the Mac, learns from you.
- **Guards:**
 - stop everything, and a step budget;
 - pausing while you use the mouse or keyboard;
 - never typing into password fields;
 - weighty clicks still ask; tiers, activity log and undo.
- **Hearing:** Parakeet, plus the speech detector.

3. The design: a speed ladder

Every request starts at the top rung and goes down only as far as it has to.

0	Direct commands. One sentence, a fixed plan, no loop. Reflex decides in about 2 ms, or one Jev call does. The details (site, name, number, text) are read from your words in code. <i>Open Chrome at an address, new tab, back, scroll, play, volume.</i>	Under 1 s after you stop talking, plus loading
1	App adapters. Each app's own interface: • browsers: tabs and addresses through Apple Events; • Finder, Mail, Messages and Music; • System Settings, through its links; • Calendar and Reminders, through Nova.app. Deterministic, and nothing to misclick.	≈ 0.1–0.5 s a step
2	Nova's own loop. Nova Eyes observes, Jev chooses, hands act, then Nova waits for the screen to settle and checks. For anything without a direct command or an adapter.	≈ 0.6–1 s a step
3	Visual fallback. Reading text and finding controls in the picture, on the Mac, for apps Accessibility can't describe.	Slower: 1–2 s a step
4	Ask you. When Nova is stuck or unsure: one short question, not a guess. Claude is optional, and off by default for computer use.	When needed

4. Rungs 0 and 1: making common commands instant

Several steps in one sentence

"Open Chrome and go to github.com, then open the pull requests" is split into steps in code, on *and* and *then*, and Reflex classifies each step. Steps Nova knows run directly. Only the unknown ones go to the loop. Each step waits for the screen to settle before the next begins.

Web addresses from speech

Spoken names become addresses in order:

- "github dot com" → github.com.
- "GitHub" → a list of known sites.
- "the Nova repo" → your bookmarks and history, read on the Mac and never sent anywhere.
- If nothing matches, a web search.

Opening the page is one Apple Event to Chrome or Safari ("open location"). If the browser is already running, that takes about 0.1–0.3 s, plus the page's own load time.

Browser controls that need no looking

New tab ⌘T, close ⌘W, next or previous tab, back ⌘[, forward ⌘], reload ⌘R, find on the page ⌘F, zoom, and the address bar ⌘L. These are instant, and the same in both browsers.

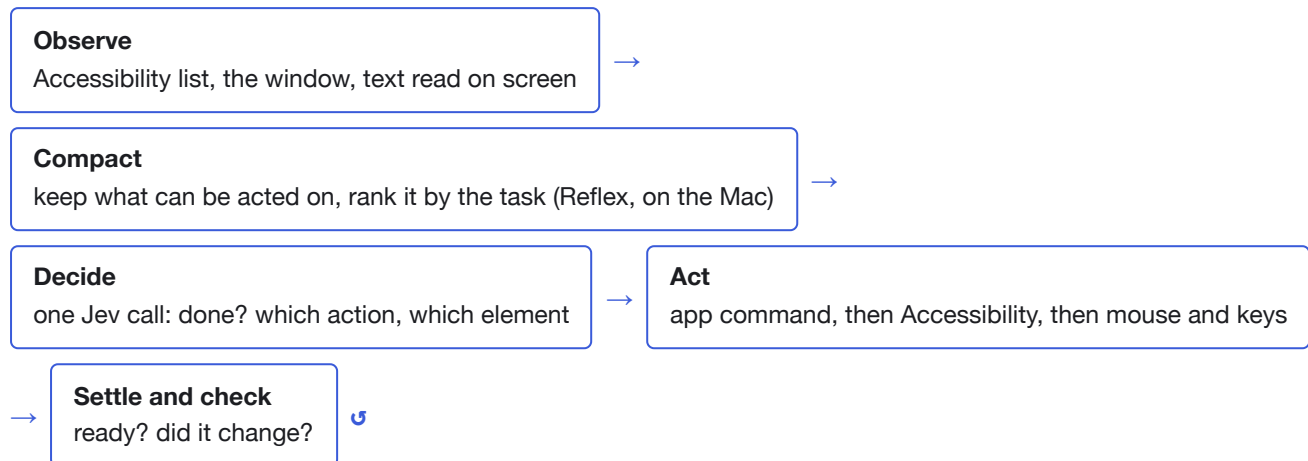
Menus by name

"Export as PDF" goes straight to the menu item through Accessibility, with no clicking around. Every Mac app's menu bar can be reached this way.

Command	Rung	Rough time after you stop talking
"Open Chrome and go to github.com"	0	≈ 0.7 s to decide and act (Chrome already open), plus loading
"New tab" / "go back" / "reload"	0	≈ 0.7 s
"Export this as PDF"	0–1	≈ 0.8 s
"Open my Agentic_OS pull requests on GitHub"	0, then 2	≈ 1 s, then 1–3 loop steps

About 0.6 s of every figure above is Nova waiting for your turn to end. That wait is a lever of its own: when a command is clear and complete ("new tab"), Nova could act sooner.

5. Rung 2: Nova's own loop with Jev



5.1 Observe

For each thing in the window, Nova Eyes already reports its role, label, value, whether it's enabled or focused, and its position. It also reports the window and app, and text read from the picture wherever Accessibility is thin. A typical app window has tens to hundreds of elements. A web page can have hundreds to thousands.

Making it fit Jev:

- Keep only what's visible and can be acted on, and fold containers into their contents.
- Rank what's left by how well it matches the task, using Reflex's word meanings on the Mac. That takes milliseconds.
- Send the top 255 at most, which is Jev's limit for one choice.
- With more than that, choose in two stages: first the part of the page, then the element.

Either way, the state stays well under Jev's 32k.

5.2 Decide: one Jev call a step, several questions at once

Jev takes several typed questions in one request, so a whole step costs one round trip:

Question	Type	Answers
Is the task done?	yes / no	Stop, and say so
What next?	choice	click · type · press keys · scroll · open · wait · ask you · give up
Which element?	choice	The ranked list, plus "none of these"
Which keys, which direction?	choice	A menu of shortcuts; up, down, left, right

Text to type never comes from Jev, because it can't write. It comes either from your words ("type see *you at five*", where the part after "type" is read in code), or from a small local model when something has to be written.

Confidence. Jev's probabilities are well calibrated. On the hardest test set, when it was 90% sure or more, it was right 98% of the time. Nova acts when Jev is sure, like today's 0.8 floor. When it isn't, Nova asks one short question ("The Pull requests tab, or Issues?").

5.3 Act

A ladder again:

1. The app's own command.
2. An Accessibility action: press it, set its value, pick a menu item, scroll it into view. None of these move your pointer.
3. Real mouse and keys, as a last resort.

Right before any click by position, Nova checks that the thing under the pointer is still the target, so an element that has moved doesn't get a stray click.

5.4 Settle and check

"Ready" means three things together, with a time limit:

- Accessibility has gone quiet: no changes to layout or values for a moment.
- Busy and loading flags have cleared.
- The picture has stopped changing, ignoring small regions such as spinners and the blinking cursor.

Then Nova checks the step worked: did what should have changed, change? If not, it looks again and retries once. If it still hasn't, it asks you.

This check is also what handles scroll-driven animation and lazy loading. Nova scrolls by element, never by distance. It waits for things to settle, then finds the target again before acting on it.

5.5 Speed per step

Observe	Decide (Jev)	Act	Settle	Per step
30–100 ms	≈ 470–520 ms (measured)	10–100 ms	0.1–0.5 s, depending on the page	≈ 0.6–1.2 s

Ways to go faster:

- **Reflex first.** When a step is obvious, for instance the named button appears exactly once, Nova acts without asking Jev. That takes about 2 ms.
- **One call per step.** All of a step's questions go to Jev together, never as separate calls.
- **Look while Jev decides.** Start reading the next screen state while Jev's answer is on its way.
- **Routines.** Replay learned sequences (section 6).
- **Push common tasks up the ladder.** Anything done often becomes a direct command or an adapter.

5.6 Living with Jev's limits

Limit	What it means	Answer
Doesn't write text	It can't write messages, or search terms you didn't say	Use your words, or a local model; otherwise ask
At most 255 options per choice	Big pages don't fit	Rank the elements; choose in two stages
At most 32k of state	Long pages don't fit	Prune, rank, and show a window of the page at a time
≈ 0.5 s per call, over the network	Every call adds half a second	Fewer calls: Reflex first, routines, direct commands
Screen text goes to TypeSafe	Privacy	• A setting to turn it off. • Passwords and secure fields are never sent. • A list of local-only apps, such as your bank or password manager. • A Reflex-only mode.
Needs the internet	No Jev while offline	Reflex takes over, as it does now, and routines still work
Cost $\approx$ \$0.0002 per call	Negligible	None needed

5.7 How good would it be?

That's unknown until measured, because Jev has never been tested on screen choices. Its record on commands is strong: 86% right on the hardest test set, against Reflex's 75%, and highly reliable when it's sure. Screens are a different job. The plan is to test it first on recorded screens, choosing without clicking anything, as your earlier agent-loop brainstorm set out.

6. Routines: doing it again, faster

When a task succeeds, Nova keeps its steps as a routine, filed under what you said:

- which app;
- which elements, by label and role, never by position;
- and what each step should change on screen.

Next time it replays the routine natively, checking at each step that the screen matches. It asks Jev only where the screen doesn't match. This is the same idea as Reflex learning from your confirmations.

A routine you use often can then be named ("do my morning GitHub check") and run like the Routines Nova already has.

7. The browser, where most of it happens

Way in	What it gives	Cost
Apple Events (Chrome and Safari)	Tabs, addresses, titles, reload. Scripts in the page only if you allow "JavaScript from Apple Events".	Nothing to install; one setting per browser
Accessibility of web pages	Links, buttons, fields and text, as the site labels them	Free; quality depends on the site
A browser extension (one for Chrome, one for Safari)	The page itself: every element, exact clicks, forms, and knowing when it's loaded or still moving	Two to build and maintain; installed once
Chrome's debugging protocol	Full control	Since Chrome 136 (2025) it won't attach to your normal profile, so it can't use your logins. Not for everyday use.

Recommendation: start with Apple Events and Accessibility, since there's nothing to install. Add an extension later for the hard pages. Chrome first, since that's what you're asking for, then Safari.

Loading and movement. An extension gives exact signals: the page has loaded, the network is quiet, the element is on screen and has stopped moving. Without one, Nova uses the settle checks in 5.4.

Animations. macOS's "Reduce motion" setting makes many sites drop their scroll animations. Nova could switch it on during a task and put it back afterwards, if you'd like that.

8. Typing and editing by voice

- **Typing.** "Type ..." types your words. Spoken punctuation ("comma", "new line") and editing ("delete that", "select the last word", "undo") come from a small grammar in code.
 - **Longer text.** Optionally, a local model writes it. Nova shows it first, and if it's about to send something, asks first.
 - **Speed.** Nova pastes for speed, then puts your clipboard back. It never types into password fields.
-

9. Stopping, and staying safe

- **Stop is instant, and needs no model.** The daemon halts, and Nova Eyes stops even partway through a step: between keystrokes, releasing any held key or button.
- **"Stop" is heard fast.**
 - Nova can act on the draft transcript, which updates every 0.7 s.
 - Or a small detector can listen for "stop" and "wait".
 - While Nova is acting, "stop" works without the wake word.
- **Three words, three meanings.**
 - "Wait" pauses.
 - "Stop" ends the task and reports what was done and what wasn't.
 - "Undo" reverses what can be reversed, and says what can't.

- **You take over.** Touching the mouse or keyboard pauses Nova at once.
 - **Guards stay.**
 - Weighty steps still ask: money, deleting, sending.
 - Text on screen is never an instruction.
 - Nova stays in the task's app.
 - Every step goes in the activity record.
 - **Misheard commands.** Nova acts when both hearing and the decision are sure. Otherwise it checks only the doubtful part. This ties in with the voice recognition brainstorm.
-

10. Showing what it's doing

Speed shouldn't feel scary. Nova shows the plan and the current step next to the orb ("2 of 3: opening Pull requests"), frames the target before acting on it, and keeps a Stop button on screen. It speaks only when it's done or needs you.

11. What could get in the way

- **Apps that don't describe themselves:** games, canvas apps, remote desktops, some Electron apps. These need the visual fallback, which is slower, or aren't supported at first.
 - **Canvas web apps** such as Figma and parts of Google Docs. They're hard without vision, even with an extension.
 - **Jev on screens.** Its accuracy there is unknown. It also depends on the network, and it raises the privacy question in 5.6.
 - **Tasks that need judgement**, like "find the cheapest flight and book it". Without Claude these are limited. Nova should say so, ask you, or let you switch a brain on for them.
 - **Permissions.** Accessibility and Screen Recording are already granted. Apple Events means one Automation prompt per app, the first time.
 - **Hearing errors become wrong actions.** The hearing work should run alongside this.
-

12. Measure before building

- **A task set.** 60–100 everyday tasks across Chrome, Safari, Finder, Settings, Mail, Music and Notes. For each task, record the screens, so Jev's choices can be tested offline without clicking anything.
- **Numbers to track:**
 - how often a task succeeds;
 - wrong actions, with none at all of the weighty kind;
 - time from the end of speech to done, at the median (p50) and the slowest 10% (p90);
 - Jev calls per task;
 - how long a stop takes.
- **Targets:**
 - direct commands under 1 s, plus loading;
 - loop steps under 1 s;

- Nova stopped within 0.3 s of hearing "stop".

13. Plan

Phase	What	Why first
1 · Instant commands	Browser adapter (addresses, tabs, getting around), menus by name, several steps in one sentence, spoken site names	The biggest speed-up you'll feel, with the least risk
2 · The native loop	Compact screen state for Jev, one call per step, native actions, settle and check, stopping mid-step, Claude off for computer use	Covers everything without a direct command
3 · Routines	Learn and replay; Reflex first; look while Jev decides	Repeated tasks get fast
4 · Browser extensions	The page itself, for hard sites	Pages that Accessibility can't see well
5 · Visual fallback	Text and controls read from the picture, on the Mac	Apps without Accessibility

The hearing work, making sure Nova hears you right, should run alongside all five.

14. Decisions for you

1. **Browser first:** Chrome, Safari, or both? (Earlier you said you use Safari; today you asked about Chrome.)
2. **Jev sees screen text at every step:** labels and values, never passwords. Is that OK? Which apps should stay local-only?
3. **Claude:** off for computer use entirely, or a last resort you can switch on?
4. **Writing text:** should Nova only type what you say, or also use a small local model, which means a download?
5. **A browser extension later:** OK to install one?
6. **"Stop" without the wake word** while Nova is acting?
7. **When Nova is sure, act at once**, multi-step tasks included, and check only when unsure or when a step is weighty?

Appendix A · The landscape, briefly

Vision-first agents read screenshots, like Claude's computer use and OpenAI's computer-using agent. They are general, but take seconds per step, and pictures of your screen go to the model's provider.

Screen parsers turn pictures into lists of elements. They're useful when an app gives no Accessibility information.

Accessibility-first agents on the Mac are the fastest wherever apps cooperate. That is Nova's advantage: Nova Eyes already produces the element list natively, so a fast chooser like Jev or Reflex is enough for most steps, and a thinking model is needed only for the rare step that takes judgement.

Appendix B · Numbers used

Number	Value	Source
Reflex, per decision	≈ 2 ms	Reflex evaluation, 27 September
Jev, per call	≈ 470 ms (p50), 520 ms (p90)	Same evaluation, about 2000 calls
Jev accuracy on the hardest set (E)	86% right, 14% wrong (Reflex: 75%, 4%, 21% to the brain)	Same
Jev when 90% sure or more	Right 98% of the time (221 of 225)	Same
Jev cost	≈ \$0.00015–0.0002 per call	≈ \$0.30 for about 2000 calls
Jev limits	32k of state; at most 255 options per choice; no free text	Agent-loop brainstorm, 28 September
End of your turn	≈ 0.24 s (speech detector) + 0.4 s (patience, finished sentence)	Nova's settings and tests
Hearing at the desk (your voice)	≈ 1% of words wrong (Parakeet, 20 scripted phrases)	Measured 28 September